CS 2143 · OBJECT-ORIENTED PROGRAMMING

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

Name _____ Date _____ Section _____

part 1 Four Ways Objects Relate

builds on W05 and W08

Most design decisions come down to one question: **how are these two classes related?** There are four basic answers. The C++ you write is what shows which one you chose.

RELATIONSHIP	MEANING	LIFETIME	TYPICAL C++	UML
Dependency (uses)	A uses a B briefly	unrelated	B is a parameter or local: `void Deal(Deck& d)`	A - - -> B
Association (knows)	A keeps track of a B it doesn't own	B outlives A	reference or pointer member: `Game& game_;`	A ——> B
Aggregation (has, shared)	A groups Bs that also exist on their own	Bs can outlive A	`vector<Player*>`, `vector<shared_ptr<B>>`	A ◇—— B
Composition (owns)	A is made of Bs	Bs die with A	value member or `unique_ptr`: `vector<Card> cards_;`	A ◆—— B
Inheritance (is-a)	A *is a kind of* B and can be used anywhere a B can	one object	`class Wizard : public Character`	A ——▷ B

QUESTION 1 — NAME THE RELATIONSHIP

Write **dependency**, **association**, **aggregation**, **composition**, or **inheritance**.

`class Hand { vector<Card> cards_; };` _____

`class ConsoleView { VideoPoker& game_; };` _____

`class Tournament { vector<Player*> entrants_; };` (players exist before and after) _____

`int Score(const Hand& hand);` as a member of `Scorer` _____

`class Car { Engine engine_; Wheel wheels_[4]; };` _____

`class Blackjack : public CardGame { ... };` _____

`class Party { vector<unique_ptr<Character>> members_; };` _____

`class Sprite { shared_ptr<Texture> texture_; };` (many sprites share one texture) _____

QUESTION 2 — SHORT ANSWER

Aggregation and composition both say “A has Bs.” What single question tells you which one it is? Answer it for a university's `Department` and its `Professor`s.

_____

_____

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 2 Is It Really “Is-A”?

Liskov substitution

Public inheritance is a promise: *a derived object can be used anywhere a base object is expected, and the code using it won't notice.* This is the **Liskov Substitution Principle** (LSP), the “L” in SOLID. “A square is a rectangle” is true in geometry. Is it true in code?

```
class Rectangle {
public:
    virtual ~Rectangle() = default;
    virtual void SetWidth(int width) {
        width_ = width;
    }
    virtual void SetHeight(int height) {
        height_ = height;
    }
    int Area() const {
        return width_ * height_;
    }
protected:
    int width_ = 0;
    int height_ = 0;
};

class Square : public Rectangle {          // keep it square!
public:
    void SetWidth(int width) override {
        width_ = width;
        height_ = width;
    }
    void SetHeight(int height) override {
        width_ = height;
        height_ = height;
    }
};

void Stretch(Rectangle& shape) {           // written by someone who only knows Rectangle
    shape.SetWidth(5);
    shape.SetHeight(4);
    cout << shape.Area() << " ";
}
```

QUESTION 3 — PREDICT AND EXPLAIN

`Rectangle r; Square s; Stretch(r); Stretch(s);` prints _____. Everything compiled and nothing crashed. So what exactly went wrong, and whose “fault” is it?

_____

_____

QUESTION 4 — GOOD IS-A, OR SHOULD IT BE HAS-A?

Write **IS-A OK** or **USE HAS-A**. For HAS-A, name the member it should be.

`class Car : public Engine` _____

`class Stack : public vector<int>` (W07 Q12) _____

`class Wizard : public Character` _____

`class Hand : public Deck` (both hold cards) _____

`class ConsoleView : public GameView` _____

`class Employee : public Person`, but people can be *re*-hired as contractors _____

Rule of thumb: “**prefer** _____ **over** _____.”

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 3 Coupling and Cohesion

Coupling is how much one class depends on the *insides* of another. Keep it **low**, so a change in one class doesn't break five others. **Cohesion** is how closely a class's methods belong together. Keep it **high**: one class, one job. Two guidelines help:

- **Law of Demeter** (“only talk to your friends”): call methods on your own members and parameters, not on objects *they* return. Chains like `a.getB().getC().doIt()` are a warning sign.
- **Tell, don't ask:** instead of pulling data out of an object and deciding for it, tell the object what you want and let it decide.

Here is part of a real FTXUI renderer for a game of War (trimmed slightly). The lambda builds the whole screen:

```
auto renderer = Renderer(buttons, [&] {
    std::string status = "Ready to play";
    if (game.isGameOver()) {
        status = "Game Over: " + game.winnerName();
    } else if (lastRound.winner == 1) {
        status = game.getPlayer1().getName() + " wins the round!";
    } else if (lastRound.winner == 2) {
        status = game.getPlayer2().getName() + " wins the round!";
    }
    if (lastRound.wasWar) {
        status += " (WAR!)";
    }
    auto player1Display = vbox({
        text(game.getPlayer1().getName()) | bold | center,
        text("Cards: " + std::to_string(game.getPlayer1().cardCount())) | center,
        renderPlayedCard(lastRound.card1) | center,
    }) | border;
    auto player2Display = vbox({ /* ... the same, for player 2 ... */ }) | border;
    return vbox({
        text("WAR") | bold | center,
        hbox({ player1Display, text("   VS   ") | center, player2Display }) | center,
        text(status) | center,
        buttons->Render() | center,
    }) | border;
});
```

QUESTION 5 — READ THE COUPLING

1. List every class or struct from the *game* side that this view has to know about.

   _____

2. Copy one line that breaks the Law of Demeter:

   _____

3. The `status` logic decides who won. Is that a *display* job or a *game rules* job? What goes wrong when you add a `--text` console view (P01 style)?

   _____

   _____

4. Suppose `Player` renames `cardCount()` to `size()`. How many places in this lambda break? _____

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 4 SOLID: Four Principles You'll Use

PRINCIPLE	IN ONE SENTENCE	YOU'VE SEEN IT
Single Responsibility	A class should have one reason to change.	P01: the game class has no I/O, and the views have no rules
Open / Closed	Open for extension, closed for modification: add features by adding code, not by editing working code.	W06: adding a `Bard` class vs. editing every `if (type == ...)` chain
Liskov Substitution	Derived objects must work wherever the base is expected.	Part 2
Dependency Inversion	Depend on abstractions (interfaces), not on concrete classes.	P01: `main` holds a `GameView`, not a `TuiView`

(The "I", Interface Segregation, says to keep interfaces small, so no class must implement methods it doesn't need.) Dependency inversion is what makes code **testable**. Compare:

```
// BEFORE: welded to randomness
class Combat {
public:
    int Damage(int bonus) {
        return rand() % 6 + 1 + bonus;
    }
};
```

```
// AFTER: depends on an abstraction
class Dice {
public:
    virtual ~Dice() = default;
    virtual int Roll() = 0;
};

class Combat {
public:
    explicit Combat(Dice& dice)
        : dice_(dice) {
    }
    int Damage(int bonus) {
        return dice_.Roll() + bonus;
    }
private:
    Dice& dice_;
};
```

```
// A "fake" for tests: replays a list
class FixedDice : public Dice {
public:
    explicit FixedDice(vector<int> rolls)
        : rolls_(rolls) {
    }
    int Roll() override {
        int value = rolls_[next_ % rolls_.size()];
        ++next_;
        return value;
    }
private:
    vector<int> rolls_;
    size_t next_ = 0;
};
```

QUESTION 6 — PREDICT AND EXPLAIN

`FixedDice dice({6, 1}); Combat combat(dice);` Then three calls to `combat.Damage(2)` return _____, _____, _____.

1. Write a test that checks a critical hit (a roll of 6, bonus 2) does 8 damage. Why couldn't you write it reliably for the BEFORE version?

   _____

   _____

2. Which part of your P01 starter code does the same job as `FixedDice`? _____

3. `Combat` stores `Dice&`. Using W08's vocabulary, what is that relationship, and who must outlive whom?

   _____

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 5 Pattern: Strategy

swap an algorithm at run time

A **design pattern** is a named, proven solution to a design problem that keeps coming up. **Strategy** pulls a decision out into its own class hierarchy, so the object that uses it can switch strategies while running. W07's `SortedList<T, Compare>` was the compile-time version. This one uses `virtual`.

```
class HitStrategy {
public:
    virtual ~HitStrategy() = default;
    virtual bool ShouldHit(int total) const = 0;
};
class Cautious : public HitStrategy {      // hits below 12
public:
    bool ShouldHit(int total) const override {
        return total < 12;
    }
};
class HouseRule : public HitStrategy { /* hits below 17 */ };
class Reckless : public HitStrategy { /* hits below 20 */ };

class Dealer {
public:
    explicit Dealer(unique_ptr<HitStrategy> strategy)
        : strategy_(std::move(strategy)) {
    }
    void SetStrategy(unique_ptr<HitStrategy> strategy) {
        strategy_ = std::move(strategy);
    }
    string Decide(int total) const {
        return strategy_->ShouldHit(total) ? "hit" : "stand";
    }
private:
    unique_ptr<HitStrategy> strategy_;
};
```

QUESTION 7 — FILL IN THE TABLE

DEALER'S STRATEGY	TOTAL 11	TOTAL 14	TOTAL 18
`HouseRule` (from the constructor)			
after `SetStrategy(make_unique<Cautious>())`			
after `SetStrategy(make_unique<Reckless>())`			

QUESTION 8 — STRATEGY VS. SUBCLASSING

Another design makes `CautiousDealer`, `HouseDealer`, and `RecklessDealer` subclasses of `Dealer`. Give two things the Strategy version can do that the subclass version can't (or can't do easily). *Hint:* the dealer gets nervous after losing three hands in a row. Also, what if betting style needs its own three variations too?

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 6 Pattern: Factory

create the right subclass from data

In W06's `makeCharacter`, a `switch` picked a subclass. Real programs pick from *data*: a save file, a config line, a menu choice. A **factory** keeps the "which class do I `new`?" decision in **one place**, so the rest of the program only ever sees the base class.

```
using Maker = function<unique_ptr<Character>(const string&)>;

unique_ptr<Character> MakeCharacter(const string& kind, const string& name) {
    static const map<string, Maker> registry = {
        {"warrior", [](const string& n) {
             return make_unique<Warrior>(n);
         }},
        {"wizard", [](const string& n) {
             return make_unique<Wizard>(n);
         }},
    };
    auto found = registry.find(kind);
    if (found == registry.end()) {
        return nullptr;
    }
    return found->second(name);
}

// Loading a party from a save file:   wizard Merlin
//                                     warrior Conan
string kind;
string name;
while (save >> kind >> name) {
    party.push_back(MakeCharacter(kind, name));
}
```

QUESTION 9 — SHORT ANSWER

1. `MakeCharacter("bard", "Jaskier")` returns _____. What happens if the loading loop pushes that into the party, and later the game calls `member->Attack()`? Suggest a better choice.

2. To add a `Bard`, list every place in the program that changes. Does the loading loop change?

3. Which SOLID principle does a factory protect, and why?

4. Why does the factory return `unique_ptr<Character>` rather than `Character` or `Character*`? (Two reasons: one from W06, one from W08.)

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 7 Pattern: Observer

"tell me when something happens"

The game knows when a score changes. A log file, a high-score banner, and an achievement tracker all care. If the game called each of them directly, it would be coupled to all of them. Instead, the **subject** keeps a list of **listeners** and calls each one when something happens. It never knows what they do.

```
class ScoreBoard {
public:
    using Listener = function<void(const string&, int)>;
    void Subscribe(Listener listener) {
        listeners_.push_back(listener);
    }
    void Record(const string& player, int points) {
        totals_[player] += points;
        for (const Listener& listener : listeners_) {
            listener(player, totals_[player]);
        }
    }
private:
    map<string, int> totals_;
    vector<Listener> listeners_;
};
```

```
ScoreBoard board;
board.Subscribe([](const string& who, int total) {
    cout << "[log] " << who
         << "=" << total << " ";
});
int high = 0;
board.Subscribe([&high](const string& who, int total) {
    if (total > high) {
        high = total;
        cout << "[HIGH " << who << "] ";
    }
});
board.Record("Rhea", 10);
board.Record("Vex", 8);
board.Record("Rhea", 5);
board.Record("Vex", 9);
```

QUESTION 10 — PREDICT THE OUTPUT

Write everything printed, in order.

_____

_____

QUESTION 11 — SHORT ANSWER

1. Add an achievement: print `[100 CLUB]` the first time anyone reaches 100. How many lines of `ScoreBoard` change? _____ Write the `Subscribe` call:

2. The second listener captures `high` **by reference**. Suppose that code were inside a function that returns, while `board` lives on. What W08 bug happens at the next `Record`?

   _____

   _____

3. FTXUI's `Button("Deal", [&] { ... })` takes a lambda to call later. Which pattern is that? _____

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 8 Pattern: Facade

one simple front door

A **facade** gives a complicated set of classes one simple interface. Callers talk to the facade only. Here it fixes Part 3's renderer: the game side offers a `WarTable` that answers one question, “what should the screen show right now?” It answers with a plain `TableView` struct, so the view never touches `Game`, `Player`, or `RoundResult` again.

```
struct SeatView {                    // plain data: everything a view needs, nothing more
    string name;
    int cardsLeft = 0;
    string lastCard;
};
struct TableView {
    SeatView left;
    SeatView right;
    string status;
    bool gameOver = false;
};

class WarTable {                     // the facade
public:
    WarTable(const string& leftName, const string& rightName)
        : game_(leftName, rightName) {
    }
    void PlayRound() {
        if (!game_.isGameOver()) {
            last_ = game_.playRound();
        }
    }
    TableView View() const {
        TableView view;
        view.left = Seat(game_.getPlayer1(), last_.card1);
        view.right = Seat(game_.getPlayer2(), last_.card2);
        view.status = Status();
        view.gameOver = game_.isGameOver();
        return view;
    }
private:
    static SeatView Seat(const Player& player, const Card& card);   // fills in one SeatView
    string Status() const;                                          // Question 12
    Game game_;                      // composition: the table owns the game
    RoundResult last_;
};
```

QUESTION 12 — MOVE THE LOGIC — WARTABLE::STATUS()

Write `Status()` as an out-of-class definition. It produces the same text the renderer used to build. (Look back at Part 3.)

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 8 Pattern: Facade (continued)

QUESTION 13 — REWRITE THE RENDERER — FTXUI OR PSEUDOCODE

Rewrite the renderer lambda using only `table.View()`. Write a helper `Element SeatBox(const SeatView& seat)` so the two player boxes aren't copy-pasted. (Pseudocode for the FTXUI calls is fine.)

QUESTION 14 — BEFORE VS. AFTER

	BEFORE (PART 3)	AFTER (FACADE)
Game-side types the view must know		
Where the “who won” text is built		
Work to add a `--text` console view		
Can you test the status text without a screen?		

A facade *hides* complexity. It doesn't remove it. Where did the complexity go, and why is that a better place for it?

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 9 Which Pattern Fits?

QUESTION 15 — MATCH EACH PROBLEM TO A PATTERN

Choose from: **Strategy**, **Factory**, **Observer**, **Facade**, **Adapter** (W07's `RankedMonster`), **RAII** (W08).

A chess AI that can play “easy”, “normal”, or “hard”, changeable mid-game _____

A level file lists enemies by name (`goblin`, `troll`), and the game must build the right objects _____

When the player's health changes, the health bar, the sound system, and the achievement tracker all need to react _____

A third-party audio library needs 6 calls to play one sound. You want `audio.Play("win")`. _____

A library's `Monster` has `Species()`, but your code expects `Name()` _____

The terminal must be restored to normal even if the game throws _____

Shipping cost can be calculated by weight, by distance, or as a flat rate, chosen at checkout _____

part 10 Design Review

put it all together

A student turns in this Blackjack design. It works. Review it the way a senior developer would.

```
class BlackjackGame {
public:
    string playerCards[12];          // e.g. "10H"
    int playerCount = 0;
    string dealerCards[12];
    int dealerCount = 0;
    Card* deck = new Card[52];
    string dealerStyle = "normal";   // "normal", "cautious", or "reckless"

    void play() {
        while (true) {
            cout << "Your cards: ";
            for (int i = 0; i < playerCount; ++i) {
                cout << playerCards[i] << " ";
            }
            cout << "\nHit or stand? ";
            string choice;
            cin >> choice;
            // ... 80 more lines: dealing, scoring aces, dealer turn ...
            if (dealerStyle == "cautious" && dealerTotal() < 12) { /* hit */ }
            else if (dealerStyle == "normal" && dealerTotal() < 17) { /* hit */ }
            else if (dealerStyle == "reckless" && dealerTotal() < 20) { /* hit */ }
        }
    }
    int dealerTotal();
    // no destructor
};
```

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

part 10 Design Review (continued)

QUESTION 16 — FIND FIVE PROBLEMS

For each problem, name the principle, guideline, or worksheet it breaks (encapsulation, SRP, OCP, Rule of Three/Zero, composition, …).

#	PROBLEM (QUOTE THE CODE)	BREAKS…
1		
2		
3		
4		
5		

QUESTION 17 — REDESIGN — CLASS SKETCH

Sketch a better design. List each class, its **one job**, its key data members (with their types, so ownership is clear), and the relationships between classes (composition, association, inheritance). Draw UML boxes and arrows if you like.

CS 2143 · OBJECT-ORIENTED PROGRAMMING

Designing with Objects — Relationships, Principles, Patterns

Review worksheet — composition vs. aggregation vs. inheritance, Liskov, coupling & cohesion, SOLID, dependency injection, Strategy, Factory, Observer, Facade — builds on Worksheets 05–08

cheat sheet Designing with Objects at a Glance

IDEA	ASK YOURSELF	C++ SHAPE
Composition	Is it *part of* me, and does it die with me?	value member, `unique_ptr` member
Association / aggregation	Do I just need to *know* about it?	reference, raw pointer, `weak_ptr`, `shared_ptr`
Inheritance	Can I be used *everywhere* my base is, without surprises? (LSP)	`class D : public B` + `virtual`
Low coupling	Am I reaching through objects? (`a.getB().getC()`)	tell, don't ask; pass plain data
High cohesion / SRP	Does this class have one reason to change?	split I/O from rules, rules from storage
Open / Closed	Can I add a variant without editing working code?	new subclass + factory, not another `else if`
Dependency inversion	Can I test this without the real screen, dice, or file?	take an interface (`Dice&`) in the constructor

PATTERN	PROBLEM IT SOLVES	SHAPE	SEEN IN
Strategy	Swap an algorithm, even at run time	object holds `unique_ptr<Strategy>`	`Dealer`, W07 `SortedList`
Factory	Build the right subclass from data, in one place	`unique_ptr<Base> Make(kind)`	`MakeCharacter`, W07 `Spawn<T>`
Observer	Many objects react to an event without the source knowing them	list of callbacks / listeners	`ScoreBoard`, FTXUI buttons
Facade	One simple interface over a complicated subsystem	wrapper class returning plain data	`WarTable`
Adapter	Make an existing class fit the interface you need	wrapper that translates calls	W07 `RankedMonster`
RAII	Never forget to release a resource	constructor acquires, destructor releases	W08 `FileHandle`, `unique_ptr`

TAKEAWAY — THE COURSE IN ONE LINE

The goal is never “more classes” or “more inheritance.” It's **useful abstractions**: each class has one clear job, a small interface, obvious ownership, and as little knowledge of other classes' insides as possible.